

HDOC – Day 30

C Programming Assignment

Name - Arushi Gusain

Sap id - 590033489

Batch 15

Date - 23/09/2026

Problem Statement

Write a C program to perform the following menu-driven operations on a given array: (1) sort the elements using Bubble Sort and perform a Binary Search, (2) perform insertion and deletion at a valid position, (3) rotate the array elements to the right by k positions, and (4) exit.

Algorithm

1. Read the number of elements and the array.
2. Display the menu repeatedly until the user selects Exit.
3. For choice 1, apply Bubble Sort in ascending order. Read the search key and perform Binary Search on the sorted array.
4. For choice 2, ask whether insertion or deletion is required. For insertion, validate the 1-based position, shift elements right and insert the new value. For deletion, validate the position, save the deleted value and shift subsequent elements left.
5. For choice 3, read k. Reduce it using $k = k \% n$ and rotate the array right by k positions using a temporary array.
6. For choice 4, terminate the program. For any other choice, display an invalid-choice message.

C Program

```
#include <stdio.h>
#define MAX 100
void display(int a[], int n) {
    int i;
    printf("Array: ");
    for (i = 0; i < n; i++)
```

```

printf("%d ", a[i]);
printf("\n");
}
void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
void rotateRight(int a[], int n, int k) {
    int temp[MAX], i;
    if (n == 0) return;
    k = k % n;
    if (k < 0) k += n;
    for (i = 0; i < n; i++)

```

```

temp[(i + k) % n] = a[i];
for (i = 0; i < n; i++)
a[i] = temp[i];
}
int main(void) {
int a[MAX], n, i, choice;
int op, pos, value, key, result, k, deleted;
printf("Enter number of elements: ");
scanf("%d", &n);
if (n <= 0 || n > MAX) {
printf("Invalid array size.\n");
return 0;
}
printf("Enter %d elements: ", n);
for (i = 0; i < n; i++)
scanf("%d", &a[i]);
do {
printf("\n--- MENU ---\n");
printf("1. Bubble Sort and Binary Search\n");
printf("2. Insertion and Deletion\n");
printf("3. Rotate array right by k positions\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
bubbleSort(a, n);
printf("Sorted ");
display(a, n);
printf("Enter element to search: ");
scanf("%d", &key);
result = binarySearch(a, n, key);
if (result != -1)
printf("%d found at position %d.\n", key, result + 1);
else

```

```
printf("%d not found in the array.\n", key);
break;
case 2:
printf("1. Insertion\n2. Deletion\nEnter operation: ");
scanf("%d", &op);
if (op == 1) {
if (n == MAX) {
printf("Array is full.\n");
break;
}
printf("Enter position (1 to %d): ", n + 1);
scanf("%d", &pos);
if (pos < 1 || pos > n + 1) {
printf("Invalid position.\n");

break;
}
printf("Enter value to insert: ");
scanf("%d", &value);
for (i = n; i >= pos; i--)
a[i] = a[i - 1];
a[pos - 1] = value;
n++;
printf("After insertion, ");
display(a, n);
} else if (op == 2) {
printf("Enter position (1 to %d): ", n);
scanf("%d", &pos);
if (pos < 1 || pos > n) {
printf("Invalid position.\n");
break;
}
deleted = a[pos - 1];
for (i = pos - 1; i < n - 1; i++)
a[i] = a[i + 1];
```

```
n--;
printf("Deleted element = %d\n", deleted);
printf("After deletion, ");
display(a, n);
} else {
printf("Invalid operation.\n");
}
break;
case 3:
printf("Enter k: ");
scanf("%d", &k);
rotateRight(a, n, k);
printf("After right rotation, ");
display(a, n);
break;
case 4:
printf("Program terminated.\n");
break;
default:
printf("Invalid choice.\n");
}
} while (choice != 4);
return 0;
}
```